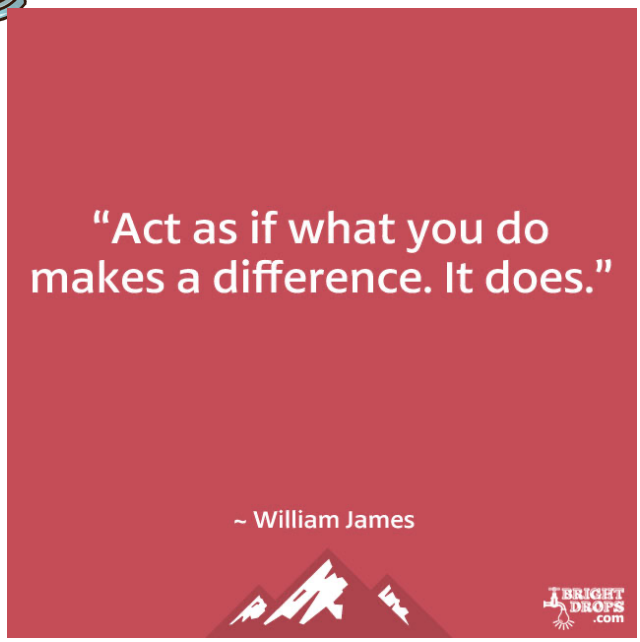
Prefix + Subarray → Interviews



Good Evening

Today's content

01. Max subarray sum
02. Zero queries : 1 & Zero queries : 2
03. Water trapping problem

01. **Max Subarray Sum**

Given arr[N] elements, return max subarr sum

subarr → continous part of arr

Constraints

$1 \le N \le 10^5$

$1 \le arr[i] \le 10^6$

Eg:- arr[3] = { -3 4 1 }

subarrays	sum
-3	-3
-3 4	1
-3 4 1	2
4	4
4 1	5
1	1

Eg:- arr[7] = { 3 2 -6 8 2 -9 4 }
0 1 2 3 4 5 6 Ans = 10

arr[7] = { -3 2 4 -1 3 -4 3 }
0 1 2 3 4 5 6 Ans = 8

Ideas → Generate all subarrays sum & get max of all sums

(a) Using 3 loops → TC = O(n³) SC = O(1)

(b) Optimised psum array → TC = O(n²) SC = O(n)

(c) Optimised cf approach → TC = O(n²) SC = O(1)

$N = 10^5$

$TC = O(10^5)^2 = 10^{10} > 10^8$

TLE

Optimization

Case 1 : If all arr[] > 0

arr[] = {4 2 1 6 7}

ans = consider complete arr as subarr

Case 2 : If all arr[] < 0

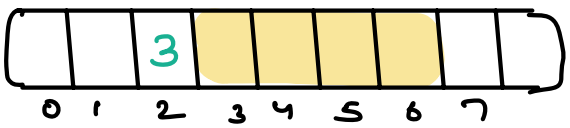
arr[] = {-4 -8 -9 -3 -5}

ans = max of the array

Case 3 : arr[] :

ans = sum of all +ve elements

Case 4 :

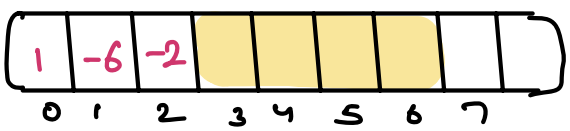


Ass 1 : [3-6] is max subarr sum

Ass 2 : arr[2] > 0

Obs :- Always take contribution of +ve ele

Case 5 :



Ass 1 : [3-6] is max subarr sum

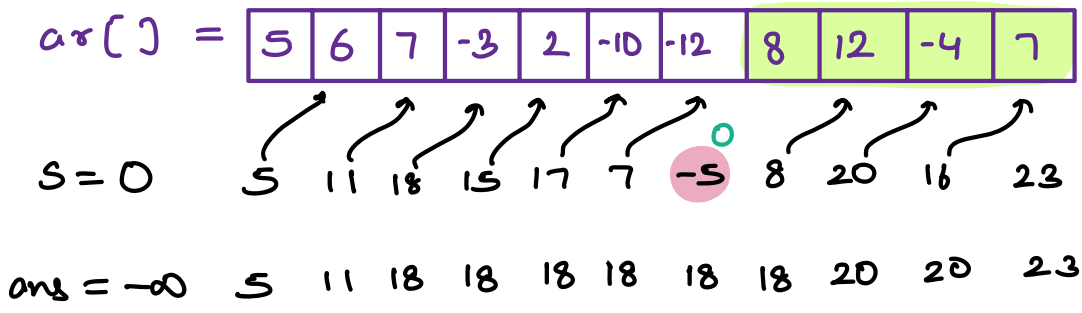
Ass 2 : arr[2] < 0

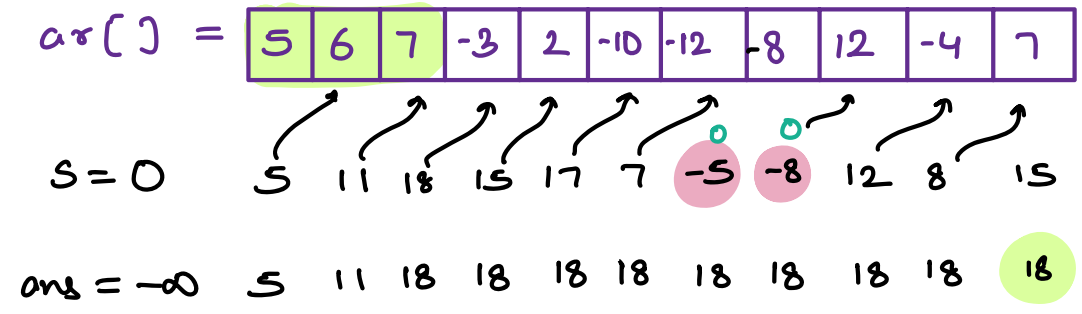
arr[1] > 0 // arr[1] + arr[2] > 0

Obs :- If a particular ele

01 +ve → definitely taking its contribution

02 -ve → sum > 0 , carry forward



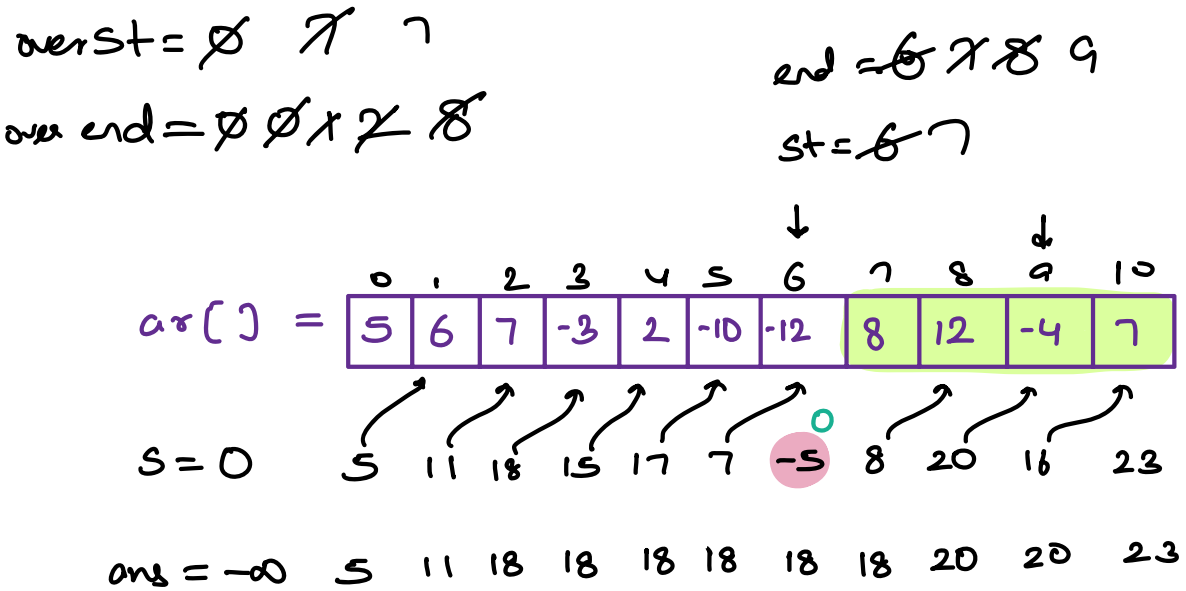



```
long subsum(int [] arr, int n)        { Kadane's Algo }
    long sum=0
    long ans= Integer.MIN_VALUE;
    for(i=0; i<n; i++){
        if(sum<0){sum=0}
        sum = sum+ arr[i];
        if(sum > ans){ans= sum};
    }
    return ans;
}
```

TC = O(n)

SC = O(1)

Return the subarray



Q2. Zero Queries

Given ar[N] = 0, all elements are zero & Q queries

For each query : Given (s, v) Add ele v to all index elements from [s...N-1]

Once all queries are done, return final arr []

Constraints

$1 \leq N, Q \leq 10^5$

$1 \leq v \leq 10^6$

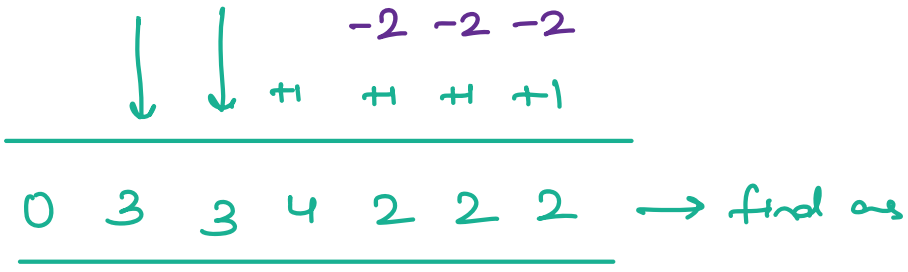
$0 \leq s < N$

Eg:- ar[7] =

0	1	2	3	4	5	6
0	0	0	0	0	0	0
+3	+3	+3	+3	+3	+3	

Queries : 3

s	v
1	3
4	-2
3	1



Idea → For every query, iterate from s to n-1 & add the value v

$TC = O(Q * N)$

$= O(N^2)$

Total iterations

$(10^5)^2 = 10^{10} > 10^8$

Code → {TODO}

Optimised Approach

ar[7] =

0	1	2	3	4	5	6
0	0	0	0	0	0	0

0 3 0 0 0 0 0

6

0 3 6 0 0 0 0

1

0 3 6 1 0 0 0

Pf ⇒ 0 3 9 10 10 10 10

Query

s	v
1	3
2	6
3	1

	0	1	2	3	4	5	
ar[4] =	0	3	-3	1	-2	0	
	0	3	0	1	-1	-1	← ans

Query : Q : 4

S	V
1	3
4	-2
3	1
2	-3

```
long [] zeroQ (int n, int s[Q], int v[Q])
    long [] ar = new long[n]

    for( i=0; i<Q ;i++){
        int st = s[i];
        int val = v[i];

        ar[st] = ar[st] + val;

    }
    Apply psum on ar;

return ar;
}
```

TC = O(Q+N)

SC = O(1)

Q **Zero Queries 2**

Given $ar[N] = 0$, all elements are zero & Q queries

For each query : Given(s, e, v) Add ele v to all index elements from [s...e]

Once all queries are done, return final arr[]

Constraints

$1 \le N, Q \le 10^5$

$1 \le v \le 10^6$

$0 \le s < N$

Ex:- ar[7] :

0	1	2	3	4	5	6
0	0	0	0	0	0	0
		2	2	2		
			3	3	3	3
		-1	-1	-1		
				2	2	2	2
0	2	4	6	4	5	2

← final arr

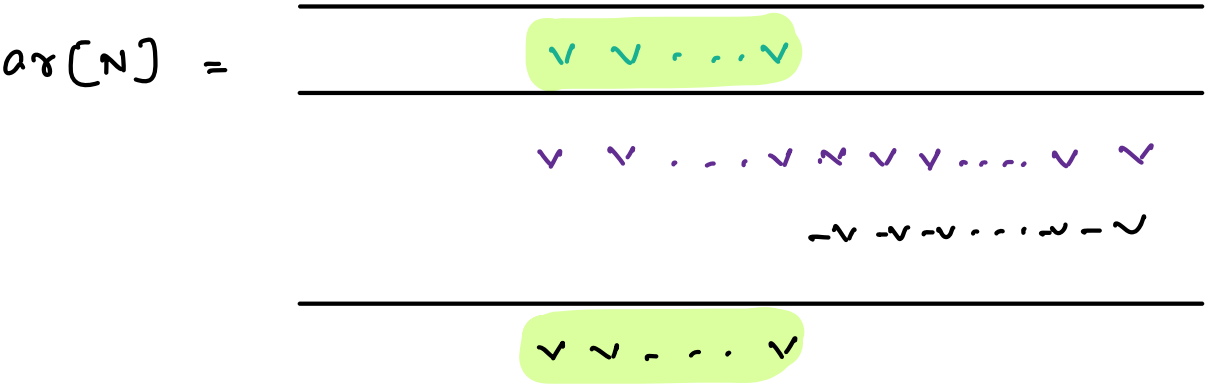
Queries

s	e	v
1	3	2
2	5	3
2	4	-1
3	6	2

Brute force :- For every query, iterate from s to e & add the value v

TC = $O(Q * n)$ = TLE

Hint :-



Query

s e +v

↳ Add +v from [s to n-1]

↳ Add -v from [e+1 to n-1]

```
long [ ] zeroQ ( int n , int s[Q] , int e[Q], int v[Q])
    long [ ] ar = new long[n]

    for( i=0 ; i<Q ; i++ ){   ⟶ TC=O(Q)
        int st = s[i];
        int end= e[i];
        int val= v[i];
        ar[st] = ar[st] + val;
        if (end+1<n){ar [end+1] = ar [end+1] - val ;}
    }
    Apply psum on ar ; ⟶ TC=O(n)   TC=O(n+Q)
                                   SC=O(1)
    return ar;
}
```

03. Given arr[N]

Create pmax[N] such that pmax[i] = max of all elements from [0...i]

eg:- arr[6] =

1	-6	3	3	8	7
0	1	2	3	4	5

pmax[6] =

1	1	3	3	8	8
0	1	2	3	4	5

```
int [] constructPmax(int [] arr)
{
    int [] pmax = new int [n];
    pmax[0] = arr[0];
    for(i=1; i<n; i++){
        pmax[i] = Math.max(pmax[i-1], arr[i]);
    }
    return pmax;
}
```

TC = O(n)
SC = O(1)

pmax[i-1] contains max of [0..i-1]

```
if(arr[i] > pmax[i-1]){ pmax[i] = arr[i];
else pmax[i] = pmax[i-1];
```

04. Given ar[N]

Create sufmax[N] such that sufmax[i] = max of all ele from [i...n-1]

	0	1	2	3	4	5	6
ar[7] :	3	10	6	7	0	2	-1
smax :	10	10	7	7	2	2	-1

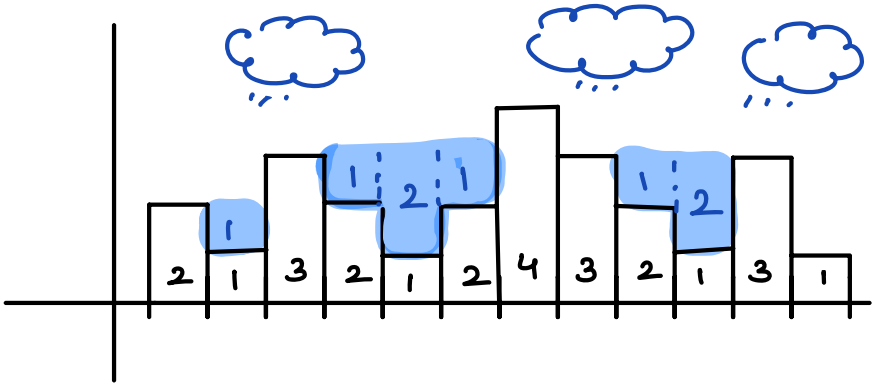
```
int [] constructsufmax (int [] arr)
    int [] smax = new int [n]
      smax [n-1] = ar[n-1];
    for (i= n-2 ; i≥0 ; i--){
        smax[i] = Math.max (smax[i+1] , ar[i]);
    }
    return smax;
}
```

05. Rain water trapped

Given ar[N] elements, where ar[i] represents height of building, return amount of water trapped on all buildings

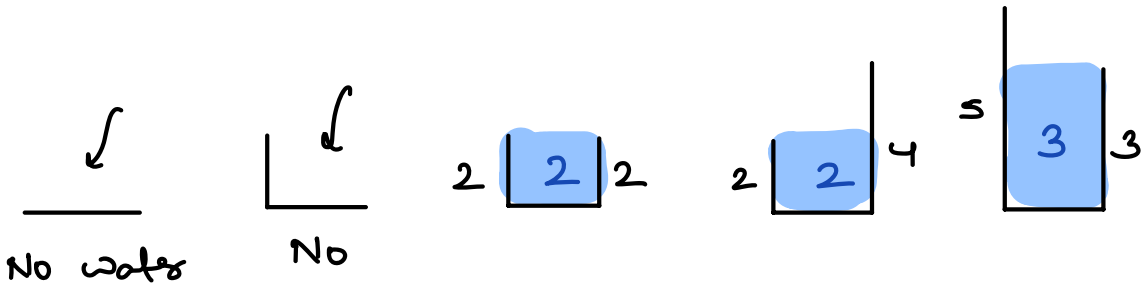
Note: Width of each building is 1

	0	1	2	3	4	5	6	7	8	9	10	11
ar[12] :	2	1	3	2	1	2	4	3	2	1	3	1

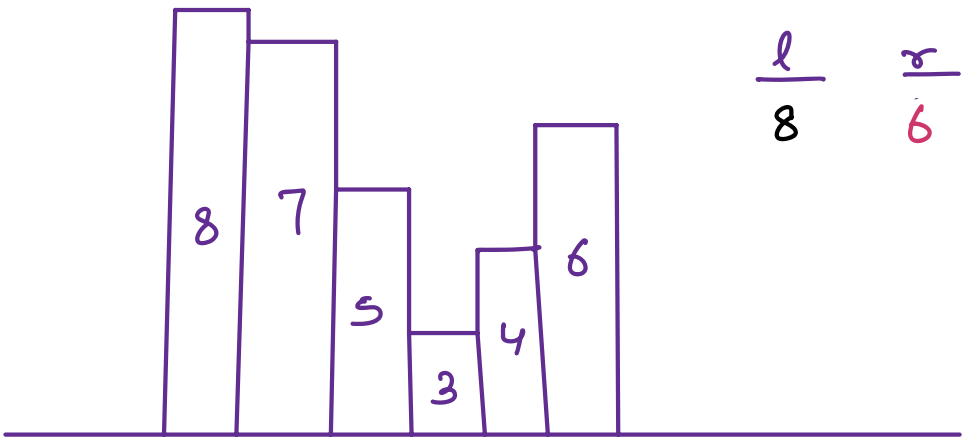


Ans = 8

Idea → Add amount of water accumulated at the top of each building



Eg:- Water accumulate on a particular building



l	r	level
8	6	6 − 3 = 3

Steps

01. Calculate left boundary : max height [0...i] → l
02. Calculate right boundary : max height [i+1 n-1] → r
03. level = Minimum (l, r)
04. Water = level − curr ht of building

arr[12] :

0	1	2	3	4	5	6	7	8	9	10	11
2	1	3	2	1	2	4	3	2	1	3	1

l → 2 2 3 3 3 3 4 4 4 4 4 4

r → 4 4 4 4 4 4 4 3 3 3 3 1

level → 2 2 3 3 3 3 4 3 3 3 3 1

ht = 2 1 3 2 1 2 4 3 2 1 3 1

water = 0 1 0 1 2 1 0 0 1 2 0 0 ⇒ 8 Ans

```
int wateracc (int [] arr)
    ans=0
    int [] sfmax = construct sufmax(arr);
    int [] pfmax = construct prefmax(arr);
      for (i=0; i<n; i++)
          l = pfmax[i]
          r = sfmax [i]
          level= Math.min (l, r):
          water= level - arr[i];
          ans += water
```

```
    }
    return ans;
}
```

TC = O(n+n+n) = O(n)

SC = O(n+n) = O(n)

Doubts

```
for (i=0 ; i<n; i++ )
{   for ( int j=i ;j<=n && j>i; j++) }
```

i	j	iteration
0	0	0
1	1	0
		0
		0
		x